Showing and explaining the C2 results

> Printable edition. Use the Contents links or PDF bookmarks to navigate. Long code lines wrap for printing; copy executable commands from the Markdown guide. Source-file links refer to the local repository; keep related PDFs together for related-guide links.

Each local file link is followed by its full path as plain text. A final `:number` identifies the source line; in VS Code, open the file with `Ctrl+P` and jump to that line with `Ctrl+G`.

Use this alongside the component study guide (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/COMPONENT_STUDY_GUIDE.md`) and metrics reference (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/METRICS_REFERENCE.md`). All commands below inspect completed results, except the clearly labelled local web-server command. None starts dataset generation, training, calibration or test evaluation. Work from `D:/VSCode/GitHub/Synth-Inversion-RP`.

About this expanded edition

This is the same walkthrough with explanations inserted at each stop. All original instructions, commands, tables and headings remain in their original order. The added explanation headings are reading notes, not extra experiments to run. You can follow either route without opening the study guide for the basic explanation.

The original opening above still points to the companion guides; those are now optional references. File paths remain visible in plain text, including source line numbers where supplied. Commands are copied unchanged; run them from the repository root. The explanations describe the saved 15 October 2026 snapshot, not a new training run or a newly evaluated test set.

Added measured tables are copied from `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/METRICS_REFERENCE.md`. The unchanged short walkthrough is `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/DEMONSTRATION_WALKTHROUGH.md`.

Contents

Choose a route or jump to a specific stage. The **Outline** panel in VS Code shows these headings and their smaller steps.

- VS Code and terminal route
- Browser demo route
- Short demonstration
- Follow-up questions

Stage	Responsibility	Jump to
Dataset generation	Shared generation pipeline	Stop 1: Shared dataset generation
Audio conditioning	Shared loader used by C2	Stop 2A: Audio conditioning with the shared utility
Train/calibration/validation/test split	C2 data preparation	Stop 2B: C2 dataset splitting
Log-mel features and normalization	C2 preprocessing	Stop 3: C2 feature preprocessing and normalization
Single CNN training and checkpoints	C2 model training	Stop 4: Single CNN training
Ensemble training and uncertainty	C2 model training	Stop 5: Ensemble training and uncertainty U
One real predicted clip	C2 saved outputs	Stop 6: Trace one saved prediction

Stage	Responsibility	Jump to
Reconstruction R and sensitivity S	C2 forward diagnostics	Stop 7A: Forward diagnostics with R and S
Correctness labels, fitting and q	C2 reliability calibration	Stop 7B: Reliability calibration and q
Metrics, intervals and robustness	C2 evaluation/validation	Stop 8: Evaluation and validation

Folder name is not the same as processing stage: `generation/corpus_audio.py` supplies the shared audio loader. C2's `v4/data.py` creates its experimental splits. Displaying `protocol.json` only shows the saved record.

Browser route shortcuts

- B1. Open the local demo
- B2. Demonstrate audio, parameters and diagnostics
- B3. Compare reliability models and examples
- B4. Show validation results and method

Before starting

Keep this sequence in mind:

```
Shared data: choose settings -> render audio -> record actual labels -> audit
C2 fitting: freeze splits -> log-mel features -> single CNN -> three probabilistic CNNs
C2 reliability: freeze predictions -> render R/S -> fit calibrators -> compare ablations
Show results: original/predicted audio -> settings -> U/S/R/q -> population evidence
```

The short opening is:

> I am testing whether forward synthesizer measurements can improve the reliability of individual parameter predictions. The original prediction stays fixed, and every reliability method judges that same prediction.

For every result, say **which examples, how many, which units and which metric**. Do not switch between the historical eight-control pilot and current 13-control v4.

Explanation: what the whole component is trying to do

Imagine hearing a machine with 13 knobs behind a curtain. First, the model guesses the knob positions. Then C2 asks how much trust to place in each guess. A knob that barely changes the sound may be difficult to recover even if the resulting patch sounds close. Conversely, a poor guess may matter a great deal acoustically.

The research question is whether two measurements made with the synthesizer add useful reliability information beyond the network's uncertainty. The main claim is not that CNNs, ensembles or finite differences are new. It is the controlled comparison of these reliability inputs for the same frozen parameter predictions.

Keep these questions separate:

Question	What answers it?
Did I recover the original knob position?	Parameter MAE/RMSE/R2, or the within-tolerance label y
Does the predicted patch resemble the target under my audio distance?	Whole-clip residual R, plus listening for illustration
How likely is one knob prediction to be within tolerance?	Per-parameter calibrated probability q

Different patches can make similar sounds. For example, with zero filter-envelope depth, changing that envelope's timing has no effect through that route. Therefore a small R does not prove all 13 original values were recovered. This experiment measures recovery of the actual generating settings, not all possible equivalent patches.

The current data is the 131,072-clip, 13-control v4 corpus. The older eight-control `pilot_v1` is historical; its 80/10/10 split and test results are not current results.

Route A: use VS Code and its terminal

Open the repository folder. `Ctrl+P` opens a file by path; `Ctrl+G` jumps to a line. For this guide, use `Ctrl+Shift+V` to open Markdown preview. The absolute file links also identify exactly which source/result is meant. The optional `code --goto` commands work when the VS Code command-line launcher is on PATH; otherwise use `Ctrl+P` and `Ctrl+G` with the same path and line.

Explanation: what to do in VS Code

Keep this expanded Markdown preview on one side, a source file or saved figure on the other, and PowerShell below. When a path ends in `:72`, open the file without that suffix using `Ctrl+P`, then press `Ctrl+G` and enter `72`.

`Get-Content` reads text; `ConvertFrom-Json` turns JSON into inspectable objects; `Import-Csv` reads a saved table; `Select-Object` chooses columns; `Format-Table` and `Format-List` only change their display. None fits a model. Opening a `.png` shows an existing plot. The `.pt` inspection below loads weights but does not train.

Stop 1: Shared dataset generation

1A. Configuration, sampling and rendering

Open these in order:

1. bend_sustain_config.py:28 (path: `D:/VSCode/GitHub/Synth-Inversion-RP/generation/bend_sustain_config.py:28`): the current parameter schema and inheritance.
2. restricted_config.py:22 (path: `D:/VSCode/GitHub/Synth-Inversion-RP/generation/restricted_config.py:22`): fixed note, rate, timing and generation seed.
3. create_sobol_samples:206 (path: `D:/VSCode/GitHub/Synth-Inversion-RP/generation/scripts/05_generate_pilot_dataset.py:206`): structured sampling.
4. apply_sampled_parameters:354 (path: `D:/VSCode/GitHub/Synth-Inversion-RP/generation/scripts/05_generate_pilot_dataset.py:354`): set knobs and record actual readbacks.
5. render_one:225 (path: `D:/VSCode/GitHub/Synth-Inversion-RP/generation/generate_corpus.py:225`): fresh synth and one complete render.

Say: "The knobs and fixed recipe define my experimental domain. Each sound has known generating settings because we controlled the synth. Sobol chooses settings; the engine readback supplies the actual label."

Explanation: the synth is the sound maker, not the CNN

Vital is the synthesizer. Vita is the Python wrapper that controls its engine. We do not recreate the synth in the browser. Shared generation code uses the engine to make supervised pairs: an audio recording and the settings that made it. Later, the same rendering recipe turns predicted settings back into audio.

The domain uses one oscillator with an embedded Basic Shapes wavetable, Bend shaping, an Analog 12 dB filter, amplitude ADSR and ENV2 routed to cutoff. Other oscillators, noise and effects stay off. Fixed conditions reduce unrelated causes of sound changes; they also limit what the trained model is demonstrated to handle.

Fixed condition	Value	Why hold it fixed?
Note and velocity	MIDI 60; velocity .8	Avoid learning varying pitch and playing strength at the same time
Timing	Note held 1.5 s; full render 3 s	Include the attack, held segment and release
Audio	44,100 Hz; canonical mono; 132,300 samples	Give every example the same shape and analysis rules
Storage	Raw FLOAT WAV masters	Preserve original samples; derive conditioning when loaded

Here is what each of the five source stops above contributes:

1. `bend_sustain_config.py` selects the current 13 controls, inheriting the earlier configuration and making filter-envelope sustain variable again.
2. `restricted_config.py` supplies fixed note/timing/rate and the restricted ranges.
3. `create_sobol_samples` draws the reproducible parameter combinations. Think of spreading dots over a 13-dimensional box. The generation seed is 20260803, and 131,072 is 2^17. Scrambled Sobol improves coverage in these chosen coordinates; it does not guarantee equal perceived timbre coverage or audible independence.
4. `apply_sampled_parameters` maps a unit sample into its allowed interval, sets the control, then reads it back. The readback, not the requested value or a rounded display string, is the canonical label.
5. `render_one` makes a fresh synth, loads the frozen preset, applies the fixed and sampled controls, renders and records the result. Fresh state and fixed phase reduce carry-over between samples. Parallel workers still commit in index order.

The 13 values and allowed engine-normalized ranges are:

Parameter	Allowed n	What it changes
wave_frame	0-1	Position in the fixed wavetable
bend_amount	.25-.75	Waveform bending; .5 is neutral
filter_cutoff	.30-.85	Main filter frequency control
filter_resonance	0-.80	Emphasis near cutoff
filter_drive	0-.75	Nonlinear filter drive
amp_attack	.05-.35	How quickly volume rises
amp_decay	.15-.40	Movement toward held volume
amp_sustain	.20-1	Held volume level
amp_release	.10-.45	Volume fall after note-off
filter_env_amount	.50-.5625	ENV2 cutoff depth; n=.5 means zero depth
filter_env_attack	.05-.35	Filter-envelope rise
filter_env_decay	.15-.40	Filter-envelope fall toward sustain
filter_env_sustain	0-1	Held filter-envelope fraction

These are normalized control positions, not directly seconds, Hz or percentages of audible change. Thirteen is the selected manageable research domain, not a proved optimal count. Ranges reflect engineering characterization and researcher review, not a completed human perceptual validation study.

Frozen preset path: `D:/VSCode/GitHub/Synth-Inversion-RP/generation/presets/base_restricted_bend_sustain_v4.vital`. Unit mapping explanation: `D:/VSCode/GitHub/Synth-Inversion-RP/generation/PARAMETER_UNITS.md`.

1B. Dataset integrity checks

Show the completed full-corpus audit without rerunning the full disk scan:

```
Get-Content -Raw .\data\processed\team_handoff_v4_131072\verification.json |
    ConvertFrom-Json |
    Select-Object verified_rows, unique_raw_audio_paths, uncheckpointed_wavs,
        raw_and_processed_hashes_verified, manifest_csv_equals_parquet_equals_checkpoints,
        quality_flags |
    Format-List
```

Expected: 131,072 rows/unique raw paths, zero uncheckpointed WAVs, matching waveform/manifest checks and zero conditioned QA flags. This displays an existing verification record; it is not a fresh full-corpus verification.

Explanation: checking the ingredients before judging the model

The audit checks that the corpus was saved consistently. Unique paths guard against accidentally reusing a file; hashes act like fingerprints of exact waveform samples; agreement between CSV, Parquet and checkpoints guards against incomplete or inconsistent bookkeeping. The recipe records synthesis settings and source/runtime identities so a later render can be checked against them.

Audio QA checks finite samples, duration/rate and levels. Silence is flagged below -60 RMS dBFS, clipping at peak >=1, and DC warnings at abs(mean)>=.005 or abs(mean)/RMS>=.20. Zero conditioned flags means these rules passed after the fixed conditioning. It does not mean all raw audio was DC-free or all sounds were perceptually good.

Reading `verification.json` shows a completed check. It does not run that check again. Whole-corpus storage/format checks may include files later reserved for test; measuring a trained model's performance on those files is a different action.

1C. Inspect one generated record

To inspect a real manifest header and the first row:

```
Get-Content .\data\raw\corpus_v4\manifest.csv -TotalCount 2
```

It is wide; use it to point out `sample_id`, `raw_audio_path`, `*_sample_01`, `*_normalized`, `*_raw`, `*_display` and hashes, not to read every value aloud. Parquet files are binary tables; VS Code's plain text view is not a table reader.

Explanation: how audio and labels stay connected

`sample_id` is the joining key. `raw_audio_path` identifies the actual source WAV; an imported prefix can remain in its original directory, so do not assume every clip is physically in one folder. The manifest is the authoritative map.

Field ending	Meaning	Use
sample_01	Proposed Sobol coordinate inside the chosen interval	Reproduce the sampling design
normalized	Actual engine-normalized readback	Canonical target label
raw	Native engine control value	Inspect units and engine behavior
display	Rounded human-readable control text	Explain, not train

For interval [lower, upper], the requested engine setting is `lower + sample_01*(upper-lower)`. Floating-point readback can differ slightly. Saving both proposal and readback avoids pretending a rounded requested value was exactly what the engine used. No human has to guess these labels from sound.

Stop 2A: Audio conditioning with the shared utility

Open corpus_audio.load_audio:48 (path: `D:/VSCode/GitHub/Synth-Inversion-RP/generation/corpus_audio.py:48`).

This function reads raw audio and applies the fixed DC policy. It lives in the shared `generation/` folder, and C2 imports it when loading audio. It does not generate new clips or decide which clips belong to training or test.

Explanation: centering the waveform without erasing level information

DC offset is a waveform shifted away from zero. The shared loader reads the raw master and applies the same causal second-order Butterworth 10 Hz high-pass once. It returns canonical float32 samples at 44.1 kHz. With verification enabled it checks raw and processed sample hashes. Tiny float32 subnormal tails are made consistent for reproducible hashes; this is not a denoising algorithm.

There is no per-clip peak or loudness normalization in this C2 path. Making every clip equally loud could remove information about amplitude-envelope or drive settings. Feature standardization at Stop 3 is a separate operation.

The key code sequence in `load_audio` is `read_raw` -> `dc_filter` -> `canonical_float32` -> optional hash checks. It does not assign splits, make new training examples or call the neural network. A shared utility can live under `generation/` without making its caller's training/splitting a generation task.

Policy path: `D:/VSCode/GitHub/Synth-Inversion-RP/generation/audio_policies/dc_highpass_10hz_v1.json`.

Stop 2B: C2 dataset splitting

Open data.assignments:72 (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/data.py:72`). This C2 function creates the train/calibration/validation/test assignments. Show protocol.json (path: `D:/VSCode/GitHub/Synth-Inversion-RP/data/processed/c2_v4/protocol_v1/protocol.json`).

The command below displays the saved C2 protocol; it does not create or change a split.

```
$c2Protocol = Get-Content -Raw .\data\processed\c2_v4\protocol_v1\protocol.json | ConvertFrom-Json
$c2Protocol | Select-Object seed, split_counts, validation_roles,
    development_prefix_training_only, diagnostic_counts, selection_policy,
    delta_z, tau_z, test_policy | Format-List
```

Explanation: separate books for learning and the final exam

Training teaches the CNN the knob positions. Calibration teaches a different, small model how to judge its errors. Inverse-validation chooses CNN checkpoints; reliability-validation chooses calibrator regularization and supports development comparisons. The final test is the sealed exam, not a tuning set.

The C2 seed is 20261013, different from generation's 20260803. In `assignments`, the already-inspected 1,024 v4-prefix IDs stay in training; a seeded permutation assigns the rest. Validation is split into two disjoint roles. A separate seed+1 ordering chooses diagnostic ranks before looking at prediction quality. Reusing that saved ordering prevents picking only convenient sounds later.

`protocol.json` records the choices. Displaying it does not perform the split. `prepare` saves the assignments and transformed labels; later stages check hashes before reusing them. It belongs to C2 at `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/data.py:132`.

Recap of audio conditioning and C2 splitting

Say: "I use a fixed DC rule once, then preserve different roles for training, reliability fitting, development selection and final testing. The previously inspected 1,024 v4 examples are training-only. Subsets were selected before looking at errors."

State the counts: 98,304 train; 16,384 calibration pool; 4,096 inverse-validation; 4,096 reliability-validation; 8,192 reserved test. Reliability uses 2,048 and 512 predetermined examples from its two pools. No model/reliability test evaluation has been performed. Whole-corpus file/format integrity checks are a separate data-QA step.

Explanation: remember the pool sizes versus the measured subsets

Role	Available clips	Actual development use
Train	98,304	CNN fitting and train-only normalization; 128 smoke diagnostics
Calibration	16,384	First 2,048 predetermined diagnostic ranks fit q
Inverse-validation	4,096	Select CNN checkpoints and report parameter errors
Reliability-validation	4,096	First 512 predetermined ranks select C and compare reliability methods
Reserved test	8,192	No model/reliability evaluation; 1,024 diagnostic IDs assigned for later

The smaller subsets bound expensive synth rendering; they are not the whole pools. The old pilot's 80/10/10 split is not this protocol. Disjoint IDs protect roles, but do not prove every sound is acoustically distinct or that this tests a new synthesizer family. Perturbations stay with their source partition and never become extra inverse-model training data in this experiment.

Stop 3: C2 feature preprocessing and normalization

Open extract_log_mel:323 (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/pipeline.py:323`), features.training_statistics:117 (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/features.py:117`) and a saved spectrogram under `D:/VSCode/GitHub/Synth-Inversion-RP/data/processed/c2_v4/baseline_full_v1/spectrogram_examples/`. Click a PNG in the Explorer to use VS Code's image preview.

```
.\venv\Scripts\python.exe -B -c 'import numpy as np;
    a=np.load("data/processed/c2_v4/features_full_v1/shard_0000.npy", mmap_mode="r"); print("Shard:", a.shape,
    a.dtype); print("One clip:", a[0].shape)'
```

Expected: shard `(512, 64, 259)`, float32; one clip `(64, 259)`. Say: "The network gets the numeric time-frequency array, not this PNG. The same training-derived mean and standard deviation are used for later inputs."

Explanation: turning sound into a small time-frequency map

A waveform is one long list of values. A spectrogram shows which frequencies are present as time moves forward. The mel filterbank groups them into 64 bands; the logarithm compresses the range of power values. The image makes this visible, but the model receives the numeric array, not the PNG, title or color palette.

Read `extract_log_mel` in this order: reflect-pad the waveform; take overlapping 2,048-sample frames every 512 samples; multiply by the Hann window; apply real FFT; scale magnitudes; square them into power; multiply by the triangular mel filterbank; take 10*log10; clip to the fixed -80 to 0 dB range; transpose to mel-by-time float32. The fixed reference is not each clip's own maximum.

Feature setting	Value
Window / hop	2,048 / 512 samples, about 46.44 / 11.61 ms
Frequency range	20-20,000 Hz
Per-clip array	64 bands x 259 time frames

Feature setting	Value
CNN input	1 x 64 x 259; a batch adds its first dimension
Cache shard	Up to 512 clips; memory-mapped to bound RAM use

`training_statistics` calculates one mean and standard deviation over training spectrogram values only. The saved values are -73.5395297540483 and 12.9035521819895. Later inputs use `(log_mel-training_mean)/training_std`, without fitting new statistics on validation or test. This keeps unseen data out of fitting.

The input is log-mel. The forward audio checks at Stop 7A use multi-resolution linear-frequency STFT magnitudes instead. Neither path currently uses CLAP or MFCC embeddings. A compact, inspectable representation is justified as a controlled baseline, not as proof of the best embedding; a stronger representation would need a matched experiment rather than an assumption that it must improve this task.

Feature configuration path: `D:/VSCode/GitHub/Synth-Inversion-RP/data/processed/c2_v4/features_full_v1/cache.json`. The full cache has 98,304 train and 4,096 inverse-validation clips; it is not a cache of evaluated test examples.

Stop 4: Single CNN training

4A. Training code and learning curves

Open ParameterCNN:82 (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/pipeline.py:82`) and baseline.train:99 (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/baseline.py:99`). Explain convolution/pooling, sigmoid outputs, MSE, backward pass and Adam using study-guide section 9. Open the learning-curve PNG beside the full baseline weights.

Explanation: the CNN learns patterns, then adjusts its own weights

A convolution is a small pattern detector sliding across time and frequency. ReLU lets the network represent nonlinear relations. Max pooling reduces the map size; average pooling and dense layers combine the learned patterns into 13 continuous outputs. This is regression, not 13-class classification.

The reused `ParameterCNN` receives 13 output bounds for v4. Importing its class from `pipeline.py` does not make this an eight-control pilot run.

```
1 x 64 x 259
 -> Conv/ReLU/MaxPool: 8 x 32 x 129
 -> Conv/ReLU/MaxPool: 16 x 16 x 64
 -> Conv/ReLU/MaxPool: 32 x 8 x 32
 -> average pool: 32 x 2 x 4
 -> flatten 256 -> dense 64 -> dense 13 -> sigmoid
```

The sigmoid constrains predictions to the research coordinate z. Actual engine positions n are converted using `z=(n-lower)/(upper-lower)` so each restricted interval spans 0-1 in the learning problem. On reconstruction, `n=lower+z*(upper-lower)`. For cutoff, z=.4 maps to n=.30+.4*.55=.52; this does not mean .52 Hz or 52% perceived brightness.

These are the central operations in the existing `baseline.train` loop:

```
optimizer.zero_grad(set_to_none=True)
prediction = model(x.to(device))
loss = torch.mean(torch.square(prediction - y.to(device)))
loss.backward()
optimizer.step()
```

`zero_grad` clears earlier gradients; `model` predicts; MSE measures disagreement with true z; `backward` calculates how weights affect that error; Adam's `step` updates the weights. Vital is not in this gradient path. The training objective is parameter error, not R and not a perceptual similarity loss.

The single CNN has 23,181 trainable weights/biases. Full training used batch 128, Adam learning rate .001, weight decay .0001, up to 60 epochs and patience 12. An epoch is one pass through the training set; patience counts consecutive epochs without validation improvement. All 60 epochs ran; the best checkpoint is epoch 53. Training and validation curves show that selection history, not a new test result.

CUDA uses a mathematically equivalent fixed 4x8 average pool to support deterministic backward execution for this input size. Fixed seeds, deterministic operations, epoch-addressable shuffle, hashes and reload checks support reproducibility in the recorded environment, not a promise of identical training on every machine.

Learning curve path:
`D:/VSCode/GitHub/Synth-Inversion-RP/data/processed/c2_v4/baseline_full_v1/learning_curves.png`.

4B. Parameter-recovery results on inverse validation

```
Get-Content -Raw .\data\processed\c2_v4\baseline_full_v1\summary.json
Import-Csv .\data\processed\c2_v4\baseline_full_v1\validation_metrics.csv |
    Where-Object coordinate -eq research_z |
    Format-Table parameter, count, mae, rmse, r_squared -AutoSize
```

Say: "Epoch 53 was selected on inverse-validation MSE. The checkpoint reload reproduced the exported predictions. My macro z MAE is .126112 on 4,096 clips." Point out strong envelope/cutoff results and weaker bend/filter-envelope results.

Explanation: reading the recovery numbers

For each parameter, let error=predicted_z-actual_z. MAE is mean(abs(error)); RMSE is sqrt(mean(error^2)), so it penalizes large mistakes more. R2 is `1-sum(error^2)/sum((actual_z-mean(actual_z))^2)`: 1 is perfect, 0 matches the constant mean fitted to those evaluated targets, and negative is possible. That R2 reference is distinct from the separately reported training-mean predictor.

A z MAE of .05 means 5% of that parameter's restricted interval on average, not 5% Hz or a guaranteed perceptual difference. Macro MAE gives each of the 13 controls equal weight. Macro RMSE is not the square root of an overall MSE; name the aggregation.

The saved single-model z MSE is .031867 versus .083213 for the training-mean baseline. The table below reports the same 4,096 inverse-validation clips for the single CNN and ensemble. The ensemble is explained at the next stop; this is not a comparison on the 512 reliability-validation examples.

Parameter	Single MAE	Single RMSE	Single R2	Ensemble MAE	Ensemble RMSE	Ensemble R2
wave_frame	0.078432	0.106992	0.861098	0.066230	0.091645	0.898087
bend_amount	0.216818	0.262532	0.165682	0.241761	0.280462	0.047828
filter_cutoff	0.056082	0.081588	0.919057	0.044266	0.067620	0.944400
filter_resonance	0.149575	0.191556	0.552953	0.156537	0.198985	0.517608
filter_drive	0.180036	0.221752	0.404213	0.215541	0.254445	0.215587
amp_attack	0.040772	0.053499	0.965886	0.015749	0.026980	0.991324
amp_decay	0.090675	0.131770	0.790556	0.076709	0.126803	0.806047
amp_sustain	0.049487	0.066501	0.947826	0.031470	0.048848	0.971849
amp_release	0.025746	0.035617	0.984816	0.003329	0.005081	0.999691

Parameter	Single MAE	Single RMSE	Single R2	Ensemble MAE	Ensemble RMSE	Ensemble R2
filter_env_amount	0.168136	0.210775	0.464376	0.186703	0.230561	0.359095
filter_env_attack	0.171434	0.213378	0.457955	0.196728	0.238382	0.323473
filter_env_decay	0.223305	0.266838	0.158404	0.240880	0.281031	0.066491
filter_env_sustain	0.188961	0.233032	0.346583	0.187984	0.233534	0.343768

Amplitude envelope and cutoff are relatively strong; bend and several filter envelope controls are harder. Possible causes include weak/conditional audio effects and model error; these scores alone do not prove a particular cause. The ensemble's macro MAE .127991 is slightly worse than the single model's .126112. Because their losses differ, this is not a causal test of averaging alone.

4C. Saved checkpoint metadata

A `.pt` checkpoint is a binary PyTorch file, not readable source. Inspect its metadata:

```
.\venv\Scripts\python.exe -B -c 'import torch;
    p=torch.load("data/processed/c2_v4/baseline_full_v1/best_model.pt", map_location="cpu",
    weights_only=True); print("Best epoch:",p["epoch"]); print("Output
    count:",p["model"]["regressor.3.bias"].shape[0])'
```

Expected: 53 and 13. This loads existing weights; it neither trains nor evaluates test.

Explanation: a trained model is its learned weights plus its recipe

The Python class defines the shape of the machine; `best_model.pt` contains its learned numbers. Loading it without the matching parameter order, feature settings and training normalization would not reproduce the same predictor. The metadata check shows the selected epoch and 13 outputs without starting inference on test.

`best_model.pt` is the selected predictor. `last_model.pt` also preserves the last training state needed for an exact configured resume. Neither is a Vital patch. A `.vital` patch stores synth settings; a `.wav` stores sound; a `.npy` stores numeric features; CSV/Parquet store tables; JSON stores configuration or results.

Model path: `D:/VSCode/GitHub/Synth-Inversion-RP/data/processed/c2_v4/baseline_full_v1/best_model.pt`. Run recipe path: `D:/VSCode/GitHub/Synth-Inversion-RP/data/processed/c2_v4/baseline_full_v1/run_config.json`.

Stop 5: Ensemble training and uncertainty U

Open GaussianCNN:29 (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/ensemble.py:29`) and combine:55 (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/ensemble.py:55`).

```
$c2Ensemble = Get-Content -Raw .\data\processed\c2_v4\ensemble_full_v1\summary.json | ConvertFrom-Json
$c2Ensemble.members | Format-Table member, seed, best_epoch, checkpoint_reload_exact
$c2Ensemble | Select-Object macro_validation_z_mae, validation_z_mse, test_evaluated
```

Say: "The model averages three parameter predictions and combines within-model variance with between-model disagreement. U is variance, not a percentage of correctness. Ensemble macro MAE is .127991, so point accuracy did not improve overall."

The complete ensemble's checkpoints are:

- member_00/best_model.pt (path: `D:/VSCode/GitHub/Synth-Inversion-RP/data/processed/c2_v4/ensemble_full_v1/member_00/best_model.pt`)

- member_01/best_model.pt (path: `D:/VSCode/GitHub/Synth-Inversion-RP/data/processed/c2_v4/ensemble_full_v1/member_01/best_model.pt`)
- member_02/best_model.pt (path: `D:/VSCode/GitHub/Synth-Inversion-RP/data/processed/c2_v4/ensemble_full_v1/member_02/best_model.pt`)

Explanation: three guesses, each with a predicted spread

Each Gaussian CNN predicts a mean setting and positive variance for every knob. The mean head uses sigmoid; the variance head uses softplus plus a 1e-4 floor. The Gaussian model has 24,026 trainable parameters and is trained with Gaussian negative log-likelihood, not the single model's MSE alone:

```
NLL = .5 * mean(log(variance) + (mean-target)^2/variance)
```

Large errors with tiny variance cost a lot, but arbitrarily huge variance also has a cost. The omitted constant does not affect gradients. NLL can be negative for small continuous variances; that is not automatically a bug. The Gaussian objective is a modelling assumption, not proof of a correct posterior.

Independent seeds are 20261013, 20262022 and 20263031; each best checkpoint is chosen using its own inverse-validation NLL. Best epochs are 57, 59 and 58. The combining code computes:

```
predicted_z = average(member_mean)
within = average(member_variance)
between = average((member_mean-predicted_z)^2)
U = within + between
```

Imagine three people estimating a knob. Each gives a range, and their central guesses may disagree. Both pieces contribute to U. The between term uses population averaging, not division by members-1. These are operational uncertainty components, not proved pure aleatoric and epistemic decompositions. Agreement can still be wrong.

U is measured in z squared. U=.02 does not mean 98% confidence. The later calibration model learns how these measurements relate to the declared correctness event. Three members are a practical bounded choice, not a demonstrated optimum. Use all three best checkpoints and the saved preprocessing settings for this ensemble.

Stop 6: Trace one saved prediction

6A. Predicted settings and diagnostics

Use the saved viewer bundle as readable JSON even without using the browser:

```
$c2Demo = Get-Content -Raw .\data\processed\c2_v4\viewer_v1\data.json | ConvertFrom-Json
$c2Example = $c2Demo.examples[0]
$c2Example | Select-Object sample_id, diagnostic_rank, R, roundtrip_exact, predictions_unchanged
$c2Example.parameters | Select-Object name, actual_z, predicted_z, error_z, U, S,
    @{Name='q_full'; Expression={$_.q.'g(U,S,R)'}} | Format-Table -AutoSize
```

Expected ID: `restricted_bend_sustain_v4_63764`; R about .098242; roundtrip and prediction-preservation true. This is one illustrative validation example, not a replacement for aggregate results.

Explanation: what is already frozen in this example

Before forward diagnostics, `predict_subsets.predict` runs the trained ensemble with fixed feature normalization, checks identities and repeated inference, then saves its means, variances and IDs. Its development selector permits smoke, calibration and reliability-validation, not test.

Prediction source path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/predict_subsets.py:42`. Viewer data path: `D:/VSCode/GitHub/Synth-Inversion-RP/data/processed/c2_v4/viewer_v1/data.json`.

The displayed example is first in the predefined reliability-validation diagnostic ordering. It was not picked for being accurate or sounding good. `actual_z` is known only because this is synthetic research data. `predicted_z` is the frozen model output; `error_z` and correctness labels judge it; U/S/R are measurements; q estimates within-tolerance recovery. Error/actual_z never become calibrator inputs.

`predictions_unchanged` verifies that diagnostics did not replace the original guess. `roundtrip_exact` says rendering the true generating settings reproduced the canonical target samples before playback encoding. It does not say the predicted settings were correct. One selected example cannot establish population performance.

6B. Listen to target and reconstruction

To play its PCM16 WAVs from the Windows PowerShell terminal inside VS Code:

```
$c2AudioFolder = Join-Path (Get-Location)
    'data\processed\c2_v4\viewer_v1\examples\restricted_bend_sustain_v4_63764'
$c2Player = [System.Media.SoundPlayer]::new((Join-Path $c2AudioFolder 'target.wav'))
$c2Player.PlaySync()
$c2Player = [System.Media.SoundPlayer]::new((Join-Path $c2AudioFolder 'predicted.wav'))
$c2Player.PlaySync()
```

Use a comfortable volume. `PlaySync()` holds this terminal until the three-second file finishes; it does not start a model. This Windows audio type and WAV loading were checked without making sound during documentation QA. If audio output is unavailable in the terminal environment, use Route B's browser players.

Explanation: why two sounds can be close while some settings are wrong

The target is the known generated clip. The reconstruction is Vital rendered at the ensemble's predicted settings, using the fixed preset/note/phase recipe. The CNN predicts controls; it does not directly generate waveform samples.

The players replay stored WAVs; they do not perform a new prediction or call a live synth. Playback copies use PCM16 for compatibility. Numerical diagnostics were computed on canonical float32 audio before quantization, without independent gain matching. Listen at a comfortable level; one person's impression is an illustration, not the later human-validation study.

6C. Inspect one parameter in detail

Show cutoff in detail:

```
$c2Cutoff = $c2Example.parameters | Where-Object name -eq filter_cutoff
$c2Cutoff | Select-Object actual_z, predicted_z, error_z, vital_normalized,
    minus_z, plus_z, actual_step_z, U, S, q | Format-List
$c2Cutoff.S * $c2Cutoff.actual_step_z
```

The last expression recovers minus-versus-plus audio distance, about .070776. Explain: z prediction .371036 maps to normalized cutoff .504070; error .001970 is inside .05; S=1.769388 is local activity; U=.001301 is variance; full q=.893669 is estimated correctness. The same clip's drive prediction is outside tolerance despite its fairly low global R.

Explanation: the cutoff arithmetic, one quantity at a time

Quantity	This saved example	Meaning
Actual / predicted z	.369066 / .371036	True setting and model guess in the restricted coordinate
Absolute error	.001970	Inside the primary tolerance .05, so y=1
Engine-normalized prediction	.504070	.30 + .371036*(.85-.30)
Lower / higher z	About .351036 / .391036	One changed knob; all other predictions fixed
Actual endpoint span	About .04	Read back from the engine, not assumed
Lower-versus-higher distance	About .070776	Difference between two diagnostic renders
S	1.769388	About .070776/.04; local acoustic activity
U	.001301	Predicted ensemble variance, in z squared
Full q	.893669	Estimated probability of recovery within .05
Whole-patch R	.098242	Target-versus-predicted render mismatch

q is not `1-error`, and it is not `1-R`. For a new unknown clip the true error is unavailable. The learned calibrator uses U/S/R to estimate the event probability. This example's drive error is .230290 with full q about .131774: several wrong knobs coexist with a fairly small global R. Filter-envelope amount succeeds with low q; one low-q success is not by itself evidence that calibration is wrong.

Stop 7A: Forward diagnostics with R and S

Open these functions in sequence:

1. Renderer.render:90 (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/diagnostics.py:90`): map settings and generate the predicted sound.
2. feature_distance:44 (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/diagnostics.py:44`): the actual audio distance.
3. perturbation_pair:62 (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/diagnostics.py:62`): central/boundary handling.
4. diagnostic_batch.measure:31 (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/diagnostic_batch.py:31`): isolation, actual span, frozen prediction and audits.

Explanation: ask the synth two different questions

First, render the frozen predicted patch and compare it with the target: that is R. Next, change one predicted knob slightly lower and higher, render both, and compare those two sounds: that gives S after dividing by actual knob separation. The target is used for R, not for deciding which perturbation to keep.

The four functions above map to the work as follows:

1. `Renderer.render` converts z to engine-normalized values, creates a fresh synth, loads the frozen preset, sets controls and checks readbacks; it renders, checks stereo agreement and applies the same canonical conditioning.
2. `feature_distance` compares linear-frequency STFT magnitudes at FFT sizes 512/1024/2048 with hops 128/256/512. Short windows resolve time more finely; longer windows resolve frequency more finely. This is not the CNN's log-mel image.
3. `perturbation_pair` changes only parameter j by delta=.02 in z. In the interior it uses z-.02 and z+.02. Near 0 or 1 it uses a one-sided pair with the original prediction at one endpoint. It never clips while pretending the span is unchanged.

4. `measure` checks the other 12 actual controls remain unchanged, uses the actual readback span, reuses the base at boundaries and verifies the prediction stayed frozen.

The diagnostic equations are:

```
SC = norm(A-B) / (.5*(norm(A)+norm(B)) + 1e-12)
LM = mean(abs(log(max(A,1e-4)) - log(max(B,1e-4))))
D = average_over_three_resolutions(SC + LM)
R = D(target_audio, predicted_audio)
S_j = D(lower_audio_j, higher_audio_j) / (actual_z_plus_j-actual_z_minus_j)
```

SC measures spectral pattern disagreement relative to magnitude; LM compares log magnitudes with a fixed floor. Both are combined by the predefined distance. R is one clip-level score shared by all parameter calibrators. It is not a validated perceptual scale: R=.1 does not mean 90% similarity.

S is an unsigned local activity measure. Low S means these two small moves have little effect under this metric; high S means a stronger effect. S is not a signed derivative of target loss, a confidence percentage, a perceptual threshold or a proof that a knob is uniquely recoverable. Other knobs can compensate for it.

We never replace the original patch with a better perturbation. Otherwise the experiment would mix better prediction with better reliability estimation.

Why not perturb every training clip? An interior patch needs up to 27 renders: one base plus two per control. For 131,072 clips that is up to 3,538,944 renders before audits. The 2,048 calibration plus 512 validation cases need up to 69,120, before boundary reuse and audits. Derived diagnostic audio is normally streamed and discarded; scores, readbacks and hashes are saved. Demo audio is retained selectively.

Stop 7B: Reliability calibration and q

1. calibration.labels:39 (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/calibration.py:39`): the within-tolerance event.
2. calibration.fit:46 (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/calibration.py:46`): the small logistic calibrator.
3. calibration.run:165 (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/calibration.py:165`): same predictions, feature ablations, validation C selection.

Explanation: teach a small judge what a correct estimate looks like

On research data we know true `z`. For each knob define `y=1 if abs(predicted_z-actual_z)<=.05 else 0`. The tolerance is 5% of that restricted parameter interval, not a sound-quality threshold. The same predictions are also examined at .025 and .10, but .05 remains the primary event.

For each parameter, a regularized logistic regression learns q from the allowed measurements. It is a small statistical model, not another large CNN:

```
Transform U with log(max(U,1e-12)); S and R with log(1+value)
Standardize features using calibration-only mean and standard deviation
score = intercept + weight_U*scaled_U + weight_S*scaled_S + weight_R*scaled_R
q = sigmoid(score) = 1/(1+exp(-score))
```

Terms absent from an ablation are left out. Log transforms compress skewed magnitudes; scaling makes coefficient fitting better behaved. Fitting uses 2,048 calibration cases with SciPy L-BFGS-B and mean binomial NLL plus `norm(weights)^2/(2*C*n)`, with an unpenalized intercept. No class rebalancing is used because the probabilities should reflect this sampled event prevalence. A single-class calibration target returns its constant observed probability.

For each knob and feature variant, C is selected from .01/.1/1/10 by lowest Brier on the separate 512 reliability-validation cases; smaller C breaks ties. Smaller C means stronger regularization. Consequently the reported validation comparisons are development estimates, not an unbiased final test.

q=.8 means that similarly scored in-domain predictions should succeed around 80% of the time if calibration holds. It is not a promise about that one prediction. There is no validated Accept/Approximate/Abstain threshold yet.

Fitted model path: `D:/VSCode/GitHub/Synth-Inversion-RP/data/processed/c2_v4/development_uncertainty_v1/calibration_tau_0.05/calibrators.json`. This saves weights, intercepts, C, scalers and feature choices so inference reuses them rather than fitting a new rule for each uploaded sound.

Recap of forward diagnostics and reliability calibration

Say: "I render at the predicted settings, not at the true settings. I move one parameter at a time and never keep a better perturbation. Separate data teaches the probability model; known error is a label, never an input feature."

Explanation: the boundary that makes this a fair experiment

The inverse model is already trained. Its predicted settings and U are frozen. R and S inspect those predictions with the synth. A separate model estimates q; it does not edit the settings. True generating values are used to teach/evaluate the correctness label, never as an input needed to calculate q on an unknown clip.

That separation is the central methodological control: all reliability methods judge the same guesses. Improvement in q scores therefore cannot be explained by quietly giving the full method a better reconstructed patch.

Stop 8: Evaluation and validation

8A. Compare reliability methods and statistical intervals

```
Import-Csv .\data\processed\c2_v4\results_audit_v1\macro_scores.csv | Format-Table -AutoSize
Import-Csv .\data\processed\c2_v4\results_audit_v1\macro_paired_comparisons.csv |
    Format-Table variant, reference, group, point_difference, ci95_low, ci95_high -AutoSize
```

Explanation: what is compared, and what each score means

An ablation removes an input while keeping the predictions, IDs and tolerance fixed. g0 is the per-knob calibration success rate. Raw U ranks likely failures without a probability calibration; it has no probability Brier/ECE. g(U) is the fair calibrated uncertainty-only baseline. g(U,S) and g(U,R) isolate additions; g(U,S,R) uses all three. Comparing full against U+R specifically asks what S adds once reconstruction mismatch is already available.

Metric	Simple explanation	Exact policy here
Brier	Squared error of a probability against a 0/1 outcome	mean((q-y)^2); lower is better; primary probability score
Reliability diagram	Compare what q says with how often predictions succeed	Plot bin mean q against observed success; diagonal is agreement
ECE	Average gap between stated probability and observed frequency	10 equal-frequency bins, weighted absolute gap; lower is better
Failure AUROC	Does a failing estimate usually get a higher failure score than a successful one?	Failure=1-y; score=1-q, or raw U; average-rank ties; higher is better
Failure AP	Precision of detected failures as more are included	Noninterpolated average precision; score ties grouped; higher is better
Coverage	Fraction of parameter predictions kept	k/n after sorting by q descending, or U ascending

Metric	Simple explanation	Exact policy here
Risk at coverage	Fraction of kept estimates outside tolerance	Cumulative failures/k; lower is better
AURC	Average retained failure risk along the coverage curve	Arithmetic mean over prefixes k=1..n; lower is better
MAE at coverage	Parameter error among estimates kept at the same rate	Cumulative absolute z error/k; lower is better

For a concrete Brier example, q=.8 with y=1 costs (.8-1)^2=.04; the same q with y=0 costs .64. An overconfident mistake costs much more. Brier measures overall probability quality, not calibration alone. ECE compares mean q with observed success in ten equal-count bins; it is bin-dependent, not proof of perfect calibration. Failure AUROC and AP use 1-q as the score (raw U for its baseline). Higher is better; AP depends strongly on the failure prevalence, so compare within a knob.

Coverage means the fraction retained after ranking by reliability. Risk is the failure fraction among retained estimates; MAE-at-coverage measures their numeric error. AURC averages prefix risks here, so lower is better. At 50% of 512 cases, each parameter retains exactly 256 predictions. Parameters may retain different audio IDs; this is not a defined whole-patch acceptance rule.

The saved macro results, with equal weight per parameter, are:

Model	Macro Brier	Macro ECE	Macro AURC
g0	0.138242	0.040681	0.568125
g(U)	0.116267	0.037460	0.462585
g(U,S)	0.115908	0.036614	0.459520
g(U,R)	0.115572	0.035921	0.462154
g(U,S,R)	0.115385	0.035282	0.459777

The full method improves Brier/ECE slightly; g(U,S) has the lowest macro AURC. No method needs to win every metric. The paired Brier differences are:

Comparison: full minus reference	Point difference	Paired 95% percentile interval
g(U), all 13	-.000882	[-.001684, -.000089]
g(U,R), all 13	-.000187	[-.000757, .000400]
g(U), exclude wave frame, exploratory	+.000024	[-.000422, .000486]
g(U,R), exclude wave frame, exploratory	+.000109	[-.000291, .000517]

The audit resamples whole audio clips with replacement 2,000 times, comparing methods on the same resampled IDs and keeping their 13 controls together. This preserves within-clip dependence. The 2.5th/97.5th percentiles give the interval. These intervals condition on fitted models and development selection; they do not include retraining uncertainty. Per-knob follow-ups are exploratory and not adjusted for multiple comparisons. Excluding wave frame is also exploratory, not a replacement for the declared 13-control comparison.

The full model's per-parameter reliability results are retained here too:

Parameter	Brier	ECE	Failure AUROC	Failure AP	AURC
wave_frame	0.183839	0.063845	0.796582	0.756097	0.266263

Parameter	Brier	ECE	Failure AUROC	Failure AP	AURC
bend_amount	0.074752	0.025909	0.818274	0.968973	0.731550
filter_cutoff	0.166348	0.039445	0.761669	0.555094	0.132704
filter_resonance	0.139402	0.042601	0.768326	0.909925	0.603304
filter_drive	0.107240	0.036229	0.607910	0.905919	0.822321
amp_attack	0.049713	0.016051	0.938763	0.452329	0.007588
amp_decay	0.169662	0.034534	0.816597	0.755776	0.188760
amp_sustain	0.119251	0.044865	0.819510	0.449437	0.055260
amp_release	0.003020	0.003348	0.790196	0.504630	0.001073
filter_env_amount	0.129552	0.042942	0.629315	0.901057	0.784369
filter_env_attack	0.136226	0.043839	0.617234	0.885317	0.769311
filter_env_decay	0.088088	0.019180	0.542121	0.914386	0.888395
filter_env_sustain	0.132908	0.045873	0.693075	0.899882	0.726209

Failure counts matter. Amplitude release has only two failures out of 512 here, so its ranking results are fragile. High AP for a mostly failing knob does not by itself imply strong discrimination. Single-class AUROC/AP are undefined.

At the same 50% coverage, the saved comparison is:

Parameter	Raw-U risk	U risk	U+S+R risk	Raw-U MAE	U MAE	U+S+R MAE
wave_frame	0.289062	0.289062	0.257812	0.037209	0.037209	0.037498
bend_amount	0.796875	0.796875	0.804688	0.132718	0.132718	0.137616
filter_cutoff	0.144531	0.144531	0.136719	0.025886	0.025886	0.025240
filter_resonance	0.656250	0.656250	0.656250	0.108292	0.108292	0.111858
filter_drive	0.855469	0.855469	0.847656	0.197020	0.197020	0.194547
amp_attack	0.000000	0.000000	0.000000	0.005521	0.005521	0.006015
amp_decay	0.164062	0.164062	0.160156	0.028808	0.028808	0.028426
amp_sustain	0.039062	0.039062	0.046875	0.017603	0.017603	0.017931
amp_release	0.000000	0.000000	0.000000	0.002521	0.002521	0.002528
filter_env_amount	0.792969	0.792969	0.804688	0.142528	0.142528	0.155169
filter_env_attack	0.800781	0.800781	0.792969	0.150692	0.150692	0.149225
filter_env_decay	0.898438	0.898438	0.890625	0.228176	0.228176	0.238795
filter_env_sustain	0.761719	0.761719	0.761719	0.153296	0.153296	0.157632

This prevents a method from gaining credit simply by keeping fewer predictions. Some controls worsen with full diagnostics. Do not replace these mixed results with a claim of uniform improvement.

8B. Perturbation step-size validation

```
Get-Content -Raw .\data\processed\c2_v4\step_stability_v1\status.json
```

Explanation: check whether the measuring ruler changes the conclusion

A very small move can be affected by numerical behavior; a larger move may no longer describe the same local neighborhood. The study checks .01/.02/.05 on the first 256 preassigned calibration ranks, with the same fixed predictions. It reuses 256 existing .02 cases and adds 512 cases for .01 and .05, plus eight audits. It does not select examples by which step gives the preferred result.

Parameter	rho .01/.02	rho .02/.05	p95 abs log1p difference .02/.05
wave_frame	0.976079	0.924167	0.689971
bend_amount	0.980507	0.938325	0.693299
filter_cutoff	0.999386	0.995610	0.215802
filter_resonance	0.999999	0.999958	0.004595
filter_drive	0.999999	0.999972	0.001051
amp_attack	0.998069	0.992270	0.404696
amp_decay	0.999987	0.999964	0.008606
amp_sustain	0.999992	0.999824	0.007125
amp_release	0.999971	0.999812	0.008518
filter_env_amount	0.999999	0.999929	0.007522
filter_env_attack	0.999621	0.998441	0.078085
filter_env_decay	0.999914	0.999752	0.006365
filter_env_sustain	0.999996	0.999917	0.003133

Spearman rho compares ordering, not magnitude. The last column is the 95th percentile of absolute differences in log(1+S); it is not a percentage or dB. Wave frame and bend are more scale-dependent. Amplitude attack has very high rank agreement but noticeably changing magnitude. Tiny boundary groups and constant scores need caution. The check did not refit calibrators or replace default .02.

Magnitude figure path:
`D:/VSCode/GitHub/Synth-Inversion-RP/data/processed/c2_v4/step_stability_v1/step_size_magnitude_agreement.png`.
Full table path:
`D:/VSCode/GitHub/Synth-Inversion-RP/data/processed/c2_v4/step_stability_v1/step_size_agreement.csv`.

8C. Implementation verification

```
Get-Content -Raw .\data\processed\c2_v4\verification_validation_viewer_v1\verification.json |
    ConvertFrom-Json | Select-Object status, tests_run, skipped, failures, errors, test_set_evaluated
```

Explanation: correct code is necessary, but is not the research result

The saved CPU suite discovered 43 tests: 41 passed and two CUDA-only checks were skipped. The GPU-only checks passed in an earlier dedicated record. Report skips honestly; a CPU run cannot establish that it ran CUDA checks.

Split protection, restart reproducibility, variance arithmetic, calibration metrics, perturbation isolation and audio exports are implementation checks. Repeated-render equality checks deterministic sound generation. Known-parameter roundtrip checks that the recorded true settings reproduce their target. Frozen prediction checks guard against diagnostics modifying the estimate. These do not prove perceptual similarity, usefulness to people or unique parameter recovery.

GPU record path: `D:/VSCode/GitHub/Synth-Inversion-RP/data/processed/c2_v4/verification_gpu_v1/verification.json`. Opening records in this walkthrough does not rerun those tests or evaluate test data.

8D. Figures and interpretation

Open per_parameter_brier_differences.png (path: `D:/VSCode/GitHub/Synth-Inversion-RP/data/processed/c2_v4/results_audit_v1/per_parameter_brier_differences.png`), matched_coverage_audit.png (path: `D:/VSCode/GitHub/Synth-Inversion-RP/data/processed/c2_v4/results_audit_v1/matched_coverage_audit.png`) and step_size_rank_agreement.png (path: `D:/VSCode/GitHub/Synth-Inversion-RP/data/processed/c2_v4/step_stability_v1/step_size_rank_agreement.png`).

Say: "Full-model Brier is .115385 versus .116267 for U alone. The conditional comparison against U+R has an interval including zero, so S's extra benefit remains uncertain. Most sensitivity ranks are consistent across steps, with important exceptions. These are development results; the final test has not been evaluated."

Explanation: how to read the saved research evidence

The Brier-difference figure shows which controls account for gains or regressions, not just an appealing average. The matched-coverage figure compares what remains when methods retain the same number. The step figure checks a measurement choice, not prediction accuracy. Read the caption, axes, population and direction before saying a bar or curve is better.

EDA means checking distributions and assumptions before treating a model score as evidence. Useful saved figures include:

Check	Why show it?	Full path
Parameter coverage and split balance	Inspect sampled ranges and non-test group comparability	`D:/VSCode/GitHub/Synth-Inversion-RP/data/processed/c2_v4/evidence_baseline_full_v1/target_distributions_and_split_balance.png`
Parameter correlation	Check sampling relationships, not acoustic independence	`D:/VSCode/GitHub/Synth-Inversion-RP/data/processed/c2_v4/evidence_baseline_full_v1/training_parameter_correlation.png`
Audio quality	Inspect levels, DC and fixed training audio conditions	`D:/VSCode/GitHub/Synth-Inversion-RP/data/processed/c2_v4/evidence_baseline_full_v1/training_audio_quality.png`
Error by parameter value	Find where recovery breaks down	`D:/VSCode/GitHub/Synth-Inversion-RP/data/processed/c2_v4/evidence_baseline_full_v1/validation_error_by_actual_value.png`
Reliability diagram	Compare q with observed success frequency	`D:/VSCode/GitHub/Synth-Inversion-RP/data/processed/c2_v4/development_uncertainty_v1/calibration_tau_0.05/reliability_diagrams.png`

The baseline evidence pack has ten figures and the ensemble pack eleven, with underlying CSVs/provenance. A low sampled-parameter correlation does not prove controls have independent acoustic effects. Plots using true envelope depth are descriptive research analysis, not inputs available for a new unknown recording.

The defensible current conclusion is a small development improvement over calibrated U, concentrated partly in wave frame. The extra contribution of S over U+R is not established by its interval. Keep that limitation visible when explaining why the research question still needs locked-method test evaluation.

Route B: use the browser demonstration

B1. Open the local demo

If the local page already loads, use that server. If it is not running, start this from the repository's PowerShell terminal:

```
.\venv\Scripts\python.exe -B -m http.server 8765 --bind 127.0.0.1 --directory data/processed/c2_v4/viewer_v1
```

Leave the terminal open. `Ctrl+C` stops this server. If the port belongs to another process, use another port instead of killing it. The complete launch/runbook is DEMO.md (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/DEMO.md`).

For a browser inside VS Code, open the Command Palette, choose **Simple Browser: Show**, and enter the URL. This is an optional viewing route; the existing local page has been tested in the Codex browser, not in every VS Code/browser build.

Explanation: the page is a saved-results viewer

The local server serves HTML, JSON, figures and audio that already exist. Starting it does not start training or a GPU job. The six examples are ordered by the preassigned validation ranks, not by how convincing they sound. q and perturbation audio were computed earlier; browser selection only changes what is displayed.

No full synthesizer has to be recreated in JavaScript for this viewer. A later live service would validate an isolated clip, apply the frozen preprocessing, run the ensemble, render with Vita/Vital, compute R/S, apply the saved calibrators and offer a compatible patch export. Upload/new-clip inference and patch-download interface remain pending. Today's page is not evidence that those are implemented.

B2. Demonstrate audio, parameters and diagnostics

Action	Explain
Start on Audio and parameters, Example 1	"This is one of six saved examples, chosen by predefined validation order."
Play Original clip, then reconstruction	"The second clip is produced by Vital from predicted settings."
Expand log-mel comparison	"Both images use the same fixed scale. The CNN receives these numeric features."
Show original settings	"Synthetic labels let us check actual recovery. A new unknown recording would not have this reference."
Select filter cutoff	"The inspector changes only this parameter for its two example renders."
Play Lower setting and Higher setting	"Their difference divided by actual setting separation gives S."
Explain U, S, q and R	"U is variance; S is local activity; R is whole-clip mismatch; q estimates within-tolerance recovery."

Explanation: what to say while moving through the audio screen

The original/predicted players answer what the two saved sounds are like. The fixed-scale spectrogram comparison shows where time-frequency energy differs; it is not a human similarity score. True settings are a research-only reference; an arbitrary unknown uploaded clip would not reveal its original controls.

Selecting cutoff shows the same base prediction and two saved perturbations: only that knob moves. The inspector does not retrain or improve the patch. For Example 1, cutoff error is .001970, S is 1.769388, U is .001301 and full q is .893669; global R is .098242. Those numbers have different units and meanings, as worked through at Stop 6C. Drive is a useful contrasting failed parameter.

Display	Plain-language explanation	Do not call it
Predicted setting	The knob position the model guessed	The guaranteed original setting
U	The ensemble's variance estimate	A correctness percentage
S	How much this knob locally changes the rendered sound	Proof the knob is uniquely identifiable
R	Whole-patch mismatch under the chosen audio distance	A similarity percentage
q	Estimated chance of being within the chosen parameter tolerance	A promise, or perceptual quality score

Do not apply an unsupported good/bad threshold to R or S. The viewer displays continuous q; Accept/Approximate/Abstain states have not been validated.

B3. Compare reliability models and examples

Action	Explain
Change Reliability model to U only, then U+R, then U+S+R	"The probabilities change, but all methods judge the same settings and sound."
Pick another example	"The examples include both successes and failures. They were not chosen for high confidence."

Explanation: change the judge, not the answer it is judging

Switching from U only to U+R or U+S+R changes the probability model. It does not change the ensemble's predicted settings, reconstructed audio or known outcome. That is the visual version of the frozen-prediction ablation at Stop 8A.

The full model can produce a higher q, a lower q, or a worse probability for an individual case. Evaluate it across the predefined population, not by whether one example's number becomes more reassuring. Use other examples to show failure cases and avoid claiming the six saved sounds represent all future recordings.

B4. Show validation results and method

Action	Explain
Open Validation results	"These aggregate 512 cases, rather than judging the research by six sounds."
Show interval and matched-coverage figures	"I compare probability quality and which errors remain at the same retained fraction."
Open Method	"This is the implemented processing order and current domain."

The browser cannot show all dataset-generation/training source code. For those questions use the file links in this walkthrough or keep the relevant source tabs open beside it. Conversely, VS Code source alone does not demonstrate how a reconstructed clip sounds; the saved WAVs or browser players do that.

Explanation: connect the demonstration to the aggregate experiment

The validation screen summarizes the 512 reliability-validation cases, not the 4,096 inverse-validation recovery set. Brier/ECE judge probabilities; AUROC/AP judge failure ranking; coverage plots judge errors among retained estimates. Stop 8A above contains their definitions and measured tables in this same file.

The key numbers to say are full Brier .115385 versus U-only .116267, a difference of -.000882. Full versus U+R is -.000187 with interval [-.000757,.000400], including zero. Thus a broad claim that sensitivity is already proven useful beyond

reconstruction mismatch would overstate the evidence. These are development results because reliability-validation also selected C. The test remains untouched.

The Method screen explains this implemented pipeline and restricted domain, not general performance on arbitrary synths, notes, effects, music or recorded instruments.

A short version if time is limited

1. Explain the question and 13-control domain.
2. Show actual-readback generation and frozen partitions.
3. Show log-mel input, CNN and single/ensemble results.
4. Play one real target/reconstruction and inspect cutoff plus a failed parameter.
5. Explain the freeze boundary, R/S, logistic q and ablations.
6. Show aggregate Brier/interval/coverage results and the remaining work.

Keep the full metrics reference (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/METRICS_REFERENCE.md`) open for per-parameter questions. Do not spend the session scrolling through raw 131,072-row manifests or relaunching long experiments.

Explanation: a compact spoken version of that same sequence

> I generated paired sounds and known settings within a fixed 13-control Vital domain. C2 keeps separate data for model fitting, reliability fitting, development selection and final testing. It converts audio to log-mel arrays and trains a CNN, then three probabilistic CNNs to provide predictions and variance. I freeze their predictions, render the patch to measure mismatch R, and perturb one knob at a time to measure local activity S. A per-knob logistic model estimates the probability of recovering the true setting within .05 of the restricted interval. I compare the same predictions under uncertainty-only and added-diagnostic variants. The development Brier gain is small; the extra benefit of S over U+R is uncertain. The saved viewer demonstrates real outputs, while live upload, final test evaluation and a small human study remain to be completed.

Use the full stop-by-stop notes above to expand whichever part needs explanation. The separate reference remains available, but the main recovery, reliability, coverage and perturbation-step numbers are now included here.

Useful answers when asked to go deeper

Why was this example selected? It is the first predefined reliability-validation diagnostic rank. The other five follow it in that same ordering.

How do you know reconstruction is implemented correctly? Known-parameter exact roundtrip, repeated-render equality, control isolation, checksummed source audio and frozen-prediction verification. Explain what each check establishes.

What would happen after uploading my own clip? The intended live service will apply supported input rules, predict settings, render a patch and compute U/S/R/q. That service/export interface remains pending. Arbitrary real-world recordings are not covered by today's calibration evidence.

Why don't all the knobs have good results? Some have weak or conditional audio effects and interactions; the estimator also has limited capacity and modelling error. The experiment measures those weaknesses and tests whether reliability information helps, rather than assuming the inverse problem is fully solvable.

What is left scientifically? Lock the final method and comparisons, evaluate the untouched test once, and later study whether a small group understands and finds the reliability display useful. No human-validation result is claimed today.

Explanation: useful distinctions behind those answers

Is this identifiability or just model difficulty? S measures local acoustic activity around a prediction. A low score can reflect an inactive route, but cannot by itself establish global non-identifiability. Estimator capacity and training error also affect recovery. The experiment tests whether measurements help judge correctness rather than claiming a complete theoretical solution.

Why logistic regression? It is small, regularized and inspectable, making it easier to compare a controlled set of U/S/R features. SciPy fits its coefficients; PyTorch fits the CNNs; NumPy handles arrays; SciPy also supports signal processing; CSV/Parquet/JSON preserve results and configuration; Matplotlib produces evidence plots; HTML/CSS/JavaScript present saved outputs. Bigger models or other embeddings are possible future comparisons, not automatically better choices.

What would the final input/output be? An isolated supported audio clip goes in; predicted controls, a compatible patch/reconstruction and per-control reliability information come out. That intended live interface is not the current static viewer. Calibration evidence is restricted to the recipe and sampled domain.

What should the small human study establish? After the method/model is fixed, use a predefined task and consent process to ask whether a small group understands the display and finds it useful. Listening judgments can complement the measured audio distance. Keep these questions separate from numerical recovery and calibration, avoid treating a convenience sample as population proof, and do not claim human-validation results before collecting them. Define the study before seeing its answers; do not use final test outcomes to repeatedly redesign the method.

What comes next scientifically? Freeze the final method, comparisons and event definition; perform the protected test evaluation once; then report its results alongside the current development evidence and the later small human study. No command in this guide initiates that evaluation.